Mawlana Bhashani Science and Technology University
Department of Information and Communication Technology

Course Title: Software Development and Project Management

Course Code: ICT 3108

StockPilot

Intelligent Inventory and Business Analytics System

Part I — Final Capstone Project Proposal and Prototyping

Student Name: **Samiul Islam**
Student ID: **IT24011**
GitHub: https://github.com/SAMIUL-01/StockPilot

Contents

1 Introduction and Background **2**

2 Problem Statement **2**

3 Aim and Objectives **2**

4 Proposed System and Unique Features **2**

5 Scope and Target Users **3**

6 Requirements **3**
6.1 Functional Requirements 3
6.2 Non-functional Requirements 3

7 Technology Stack and Architecture **3**

8 Software Process Model **3**

9 Schedule, Milestones and Deliverables **4**

10 UML Diagrams **4**
10.1 Data Flow Diagram 4
10.2 Use Case Diagram 5
10.3 Activity Diagram 6
10.4 Entity Relationship Diagram 6
10.5 Class Diagram 7
10.6 Object Diagram 8

11 Prototype **9**

12 Testing Plan and Expected Outcomes **9**

13 Limitations and Future Enhancements **9**

Abstract

Small retail shops in Bangladesh usually track stock in notebooks or spreadsheets. This causes stock-outs, expired or dead stock, and no clear view of profit. **StockPilot** is an inventory management system that goes beyond storing stock counts. It records every stock movement, calculates gross profit per sale, and gives rule-based *Smart Restock* advice explaining what to reorder, how much, and why. This proposal presents the problem, objectives, scope, requirements, process model, schedule with Gantt chart, six UML diagrams, and a working Java prototype with a simple Swing interface built from seven classes.

1 Introduction and Background

Inventory is the largest working asset of a small retail business. Wrong stock decisions either tie up money (over-stocking) or lose sales (stock-outs). Existing commercial systems are expensive or too complex for small shops. StockPilot targets a low-cost, easy-to-use system that turns daily sales data into reorder decisions.

2 Problem Statement

- Stock is tracked manually, so counts are often wrong.
- Owners only notice low stock after a product has run out.
- Profit per product is unknown because cost and selling price are not linked to each sale.
- There is no history of who changed stock, when, and why.

3 Aim and Objectives

Aim: to design and prototype an inventory system that supports daily operations and recommends restocking decisions.

1. Manage products, categories and suppliers with validated data.
2. Process sales with automatic invoice calculation and stock deduction.
3. Maintain a complete audit trail of every stock movement.
4. Compute gross profit and margin from cost snapshots stored with each sale.
5. Recommend reorder quantity using sales history and supplier lead time.
6. Provide a dashboard showing KPIs and low-stock alerts.

4 Proposed System and Unique Features

Smart Restock Advisor Reorder point $= \lceil \text{avg daily sales} \times \text{lead time} \rceil + \text{safety stock}$. If current stock is below it, the system suggests an order quantity and prints the reason. Example: average 5/day, lead time 4 days, safety stock 8 gives reorder point 28; with stock 12 an alert is raised.

Profit Intelligence Each sale line stores a price and cost snapshot, so profit stays correct when prices change later.

Stock Movement Audit Trail Every stock-in, sale and adjustment is logged with time, type, quantity and reason.

All-or-nothing billing A sale is validated completely before any stock is deducted; insufficient stock rejects the whole invoice.

Expiry and dead-stock reports Planned for the full version.

5 Scope and Target Users

In scope (prototype): product management, sales and billing, low-stock alerts, restock advice, audit trail, dashboard. **Planned (full version):** purchases, supplier management, user roles, reports export, expiry tracking. **Out of scope:** online payment, e-commerce integration, multi-branch support. **Users:** Admin (shop owner/manager) and Staff (sales counter).

6 Requirements

6.1 Functional Requirements

FR1. The system shall allow Admin to add products with unique SKU, cost, price, opening stock and reorder level.
FR2. The system shall allow Staff to build a cart and complete a sale.
FR3. The system shall reject a sale if any item exceeds available stock.
FR4. The system shall deduct stock and log a movement for every sale.
FR5. The system shall flag products whose stock is at or below reorder level.
FR6. The system shall show restock suggestions with quantity and reason.
FR7. The system shall calculate total sales and gross profit.
FR8. The system shall support role-based login (full version).

6.2 Non-functional Requirements

Usability: simple dashboard, usable without training. **Reliability:** no negative stock; stock and sale updated together. **Performance:** actions respond within one second for up to 10,000 products. **Maintainability:** separate classes with single responsibilities. **Security:** hashed passwords and role-based access (full version). **Portability:** runs on any system with JDK 21.

7 Technology Stack and Architecture

Layer	Technology
Prototype UI	Java Swing (JDK 21)
Business logic	Plain Java (OOP), packages `stockpilot`
Full-version backend	Java, Spring Boot, Maven (REST API)
Full-version frontend	HTML, CSS, JavaScript, Chart.js
Database	MySQL (prototype uses in-memory collections)
Tools	Git/GitHub, diagrams.net, Graphviz, JUnit, Overleaf

The prototype follows a layered design: UI (`MainApp`) $\rightarrow$ service layer (`InventoryService`, `RestockAdvisor`) $\rightarrow$ domain classes (`Product`, `Sale`, ...).

8 Software Process Model

Selected model: Agile (Scrum-style incremental development). Requirements are clear at a high level but details (reports, restock rules) will change after feedback, so short iterations suit the project better than a rigid Waterfall plan. Work is divided into sprints, each ending with a working increment.

Sprint	Period	Goal
Sprint 0	18–21 Oct	Requirements, design, UML, repository setup
Sprint 1	22–24 Oct	Core classes, sales workflow, Swing interface
Sprint 2	25–27 Oct	Testing, screenshots, report, submission

After submission, further sprints add purchases, user roles, Spring Boot API with MySQL, reports and deployment.

9 Schedule, Milestones and Deliverables

Task	Dates (2026)	Deliverable
Scope and requirements	18 Oct	Requirement list (M1)
Proposal draft	19–20 Oct	Proposal sections
UML and database design	19–21 Oct	Six diagrams, ERD (M2)
Project setup, class design	21 Oct	GitHub repo, class skeleton
Core classes	22–23 Oct	Working domain and service code (M3)
Swing prototype	23–24 Oct	Dashboard and workflow screens
Testing and screenshots	25 Oct	Test results, screenshots (M4)
Report finalization	25–26 Oct	Final PDF, README
Review and submission	27 Oct	Repository URL on Canvas (M5)

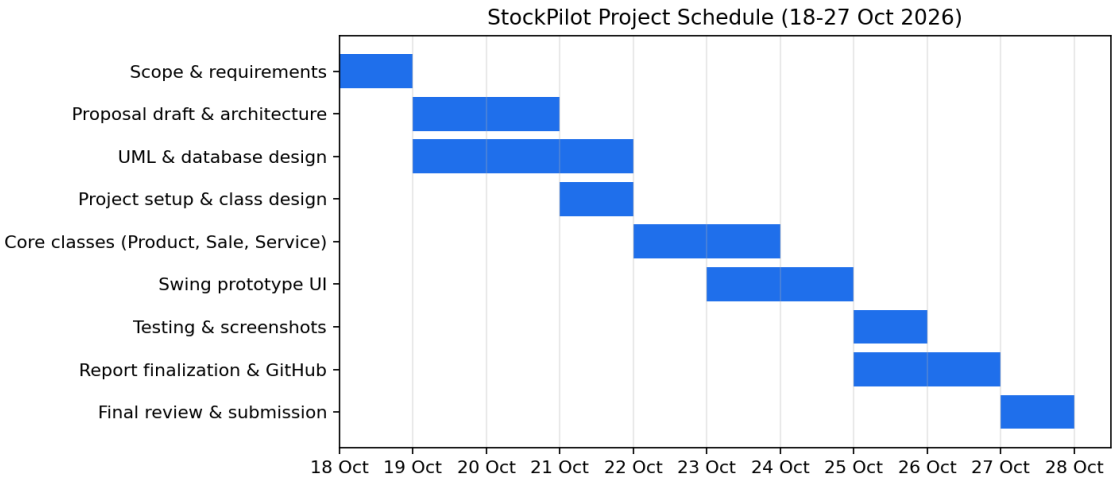


Figure 1: Gantt chart

10 UML Diagrams

10.1 Data Flow Diagram

Admin, Staff and Supplier interact with five processes and four data stores. Sales deduct stock and write stock movements; the restock advisor reads sales history and stock.

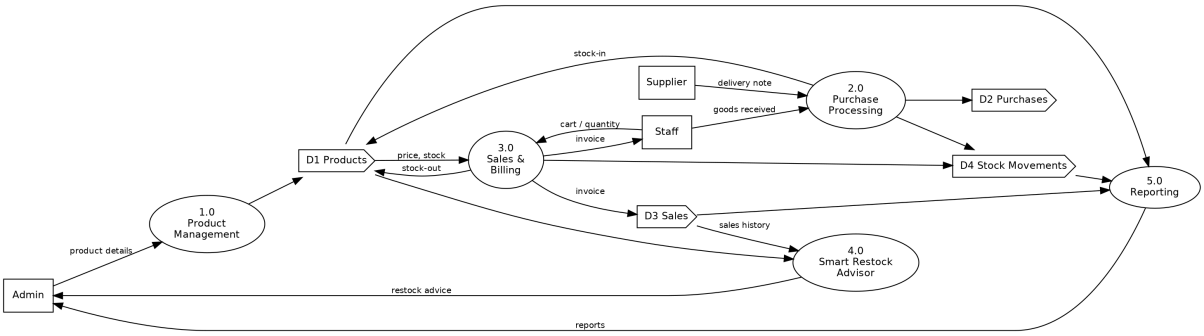


Figure 2: Data flow diagram

10.2 Use Case Diagram

Admin manages products, suppliers, users and views reports; Staff records purchases and processes sales.

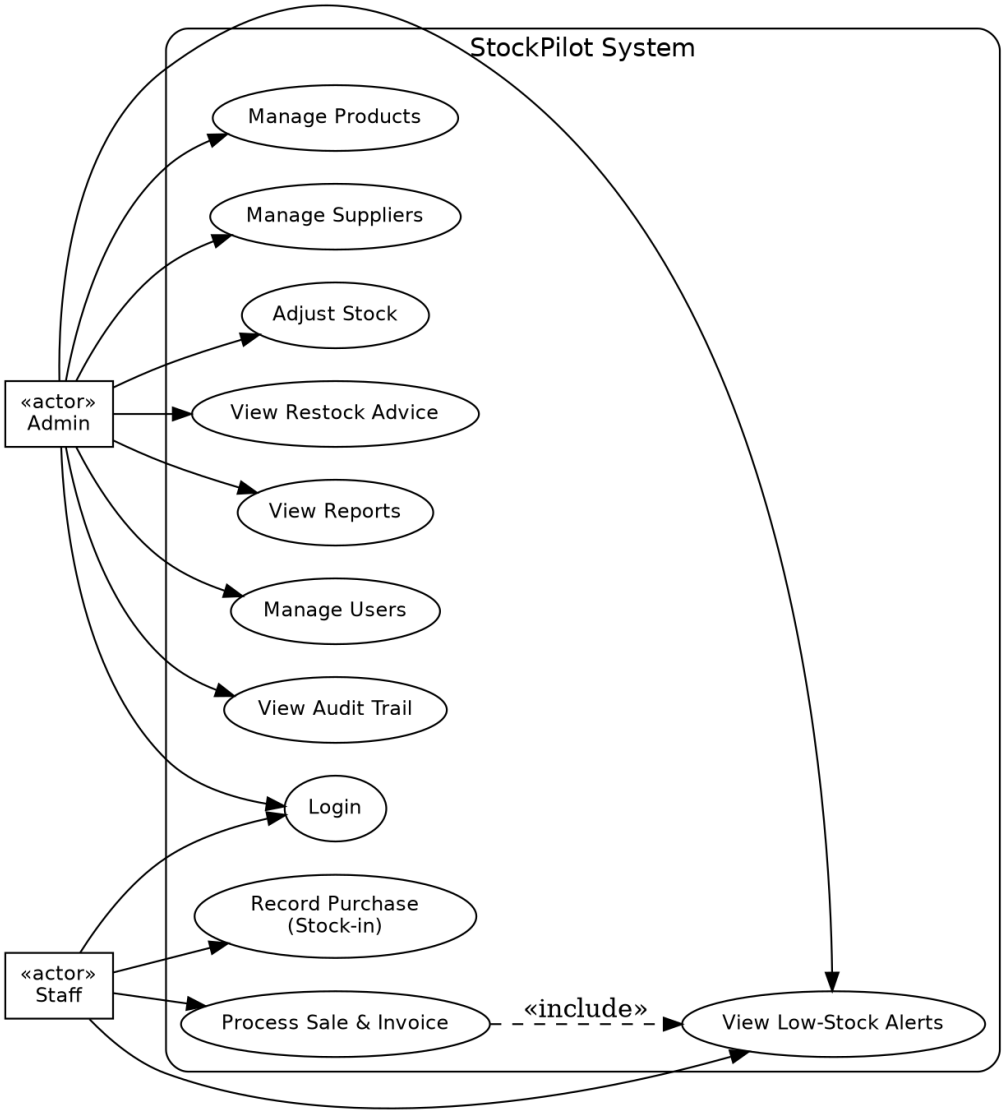


Figure 3: Use case diagram

10.3 Activity Diagram

The sales workflow, including the error branch for insufficient stock.

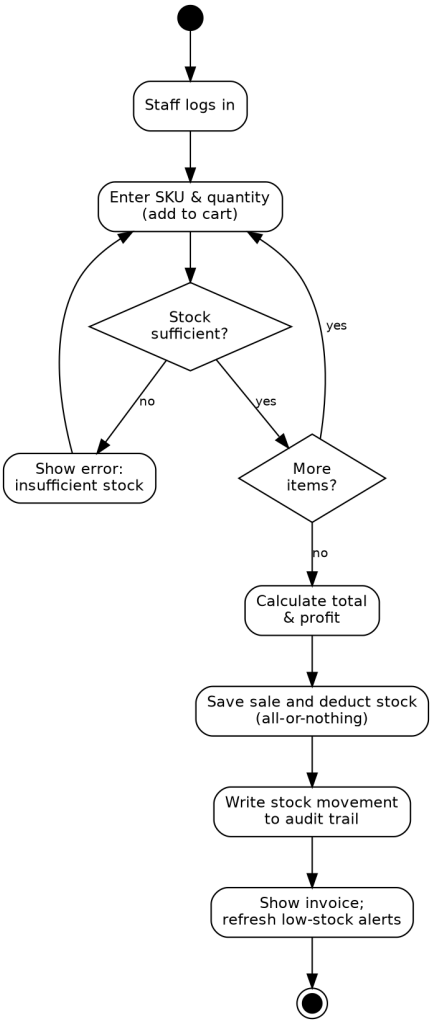


Figure 4: Activity diagram: sales and billing

10.4 Entity Relationship Diagram

Nine tables for the full version; `stock_movements` records every stock change.

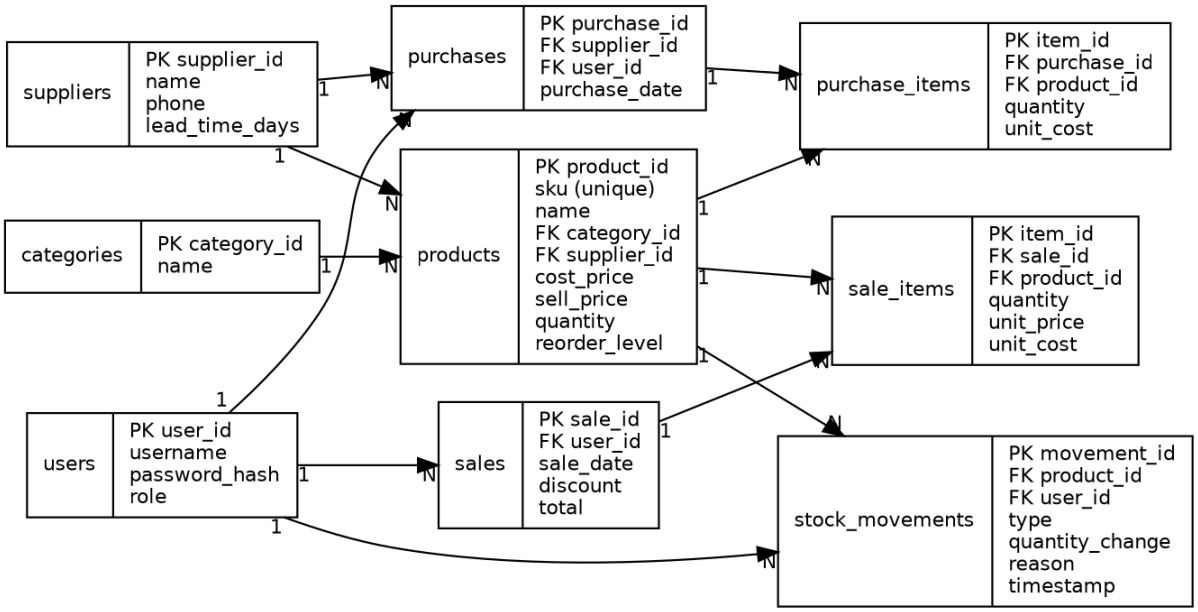


Figure 5: Entity relationship diagram

10.5 Class Diagram

The prototype implements these classes: `Category`, `Supplier`, `Product`, `SaleItem`, `Sale`, `InventoryService`, `RestockAdvisor`, `InsufficientStockException` and the UI class `MainApp`.

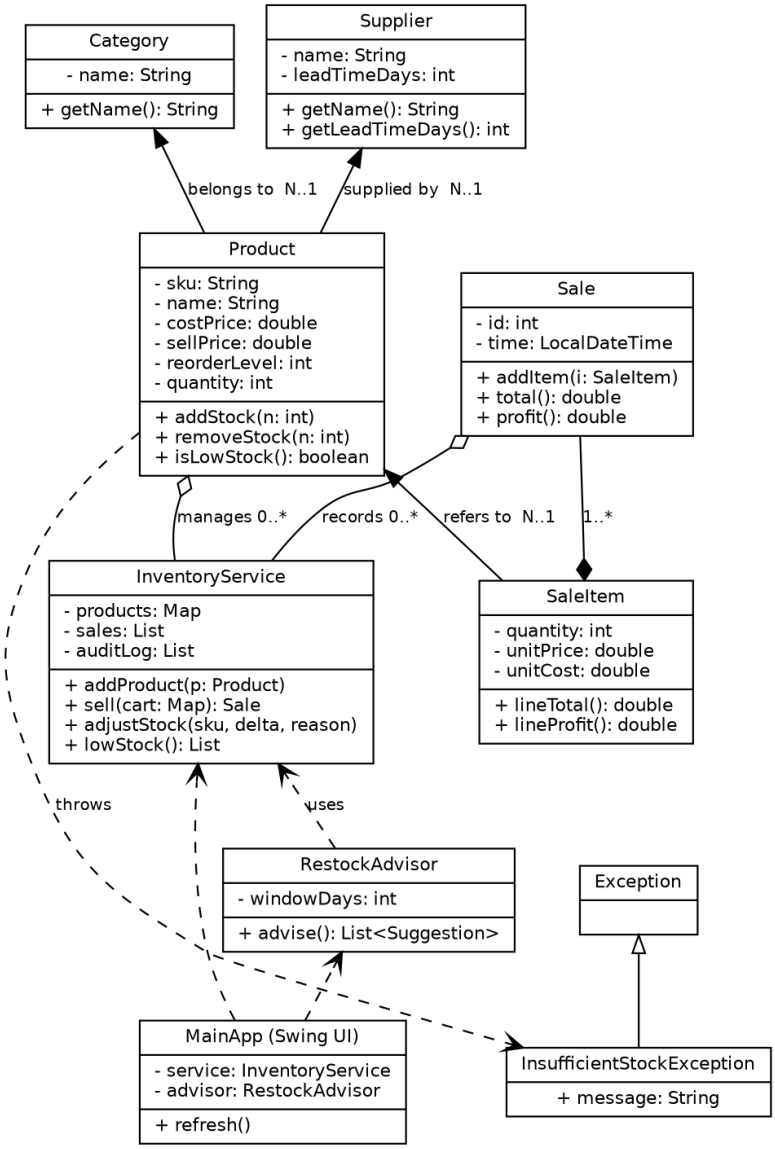


Figure 6: Class diagram

OOP concepts used: encapsulation (private `quantity` changed only through validated methods), composition (`Sale` contains `SaleItems`), inheritance (custom exception extends `Exception`), exception handling (rejecting oversized sales), and abstraction through the service layer.

10.6 Object Diagram

A snapshot of sample objects at run time.

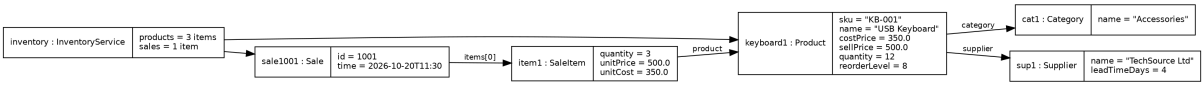


Figure 7: Object diagram

11 Prototype

The prototype is a Java Swing desktop application in `src/main/java/stockpilot`. It uses seven domain/service classes plus the UI. Tabs: **Dashboard** (product count, sales, low-stock count, gross profit), **Products** (table and add form with validation), **Sales and Billing** (cart, checkout, invoice), **Smart Restock** (suggestions with reasons), **Audit Trail**. Data is held in memory; database persistence is planned. Run instructions are in the README.

12 Testing Plan and Expected Outcomes

Test	Input	Expected result
Add product	Valid SKU, prices, stock	Product listed, audit entry added
Duplicate SKU	Existing SKU	Error message
Valid sale	KB-001 $\times$ 3	Stock reduced by 3, invoice shown
Oversized sale	KB-001 $\times$ 100	Rejected, stock unchanged
Low-stock alert	Stock $\leq$ reorder level	Product marked LOW
Restock advice	Stock below reorder point	Suggestion with reason

Expected outcome: a working, validated inventory workflow that demonstrates the design and can be extended into the full system.

13 Limitations and Future Enhancements

The prototype stores data in memory, so it resets on exit; login, purchases, expiry alerts, exports and the Spring Boot/MySQL backend are future work. Restock advice uses simple averages and does not yet consider seasonality or pending orders.

References

[1] I. Sommerville, *Software Engineering*, 10th ed., Pearson, 2016.

[2] M. Fowler, *UML Distilled*, 3rd ed., Addison-Wesley, 2003.

[3] K. Schwaber and J. Sutherland, *The Scrum Guide*, 2020.

[4] Oracle, *Java SE Documentation*, https://docs.oracle.com/en/java/.

GitHub Repository

https://github.com/SAMIUL-01/StockPilot